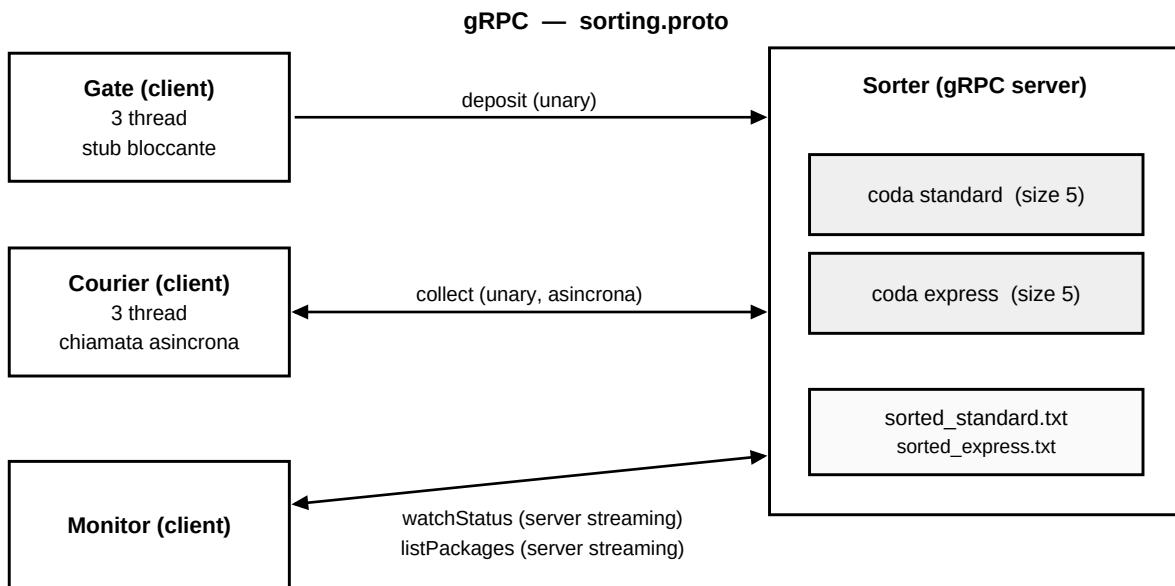


Università degli Studi di Napoli Federico II
Advanced Computer Programming
Esercitazione 06
gRPC

Il candidato realizzi un'applicazione per la gestione di un **centro di smistamento pacchi**, in cui la comunicazione tra tutte le entità è realizzata **esclusivamente attraverso gRPC**. Il sistema smista **pacchi**, ciascuno caratterizzato da un *id* (intero), che lo identifica univocamente, da una *priority* (stringa), che indica se si tratta di una spedizione *standard* o *express*, e da una *destination* (stringa), scelta tra "NA", "RM" e "MI". Il sistema è composto da 4 entità, come illustrato in figura.



Sorter è un server gRPC che implementa i servizi descritti dal file `sorting.proto` riportato in fondo alla traccia. Il server deve essere configurato con un **pool di thread di dimensione pari a 3**, in modo da poter servire più client contemporaneamente. Il Sorter possiede **2 code** a gestione circolare, una per ciascuna *priority*, entrambe di dimensione pari a 5: l'accesso alle code è disciplinato secondo il problema **produttore/consumatore**. Il server espone i seguenti metodi:

- **rpc deposit(Package) returns (Ack)**: riceve un pacco e ne inserisce l'id nella coda indicata dal campo *priority*; se la coda è piena l'inserimento è messo in attesa. Il metodo restituisce l'esito dell'operazione, e.g., `{"result": "deposited"}`. Da notare che l'operazione deve fallire nel caso in cui venga indicata una *priority* non prevista: in tal caso il servente non restituisce un **Ack**, ma termina la chiamata con un **errore gRPC** (e.g., `INVALID_ARGUMENT`), che il cliente deve intercettare e gestire.
- **rpc collect(CollectRequest) returns (Package)**: preleva un pacco dalla coda indicata dal campo *priority* della richiesta; se la coda è vuota l'operazione è messa in attesa. Il Sorter salva su file (`sorted_standard.txt` oppure `sorted_express.txt`) l'id del pacco prelevato e l'identificativo del corriere richiedente, quindi restituisce il pacco al chiamante.
- **rpc watchStatus(Empty) returns (stream Status)**: restituisce al chiamante uno **stream** con lo stato del sistema. Ogni 2 secondi il Sorter invia un messaggio *Status* contenente il numero di pacchi attualmente presenti in ciascuna delle due code ed il numero totale di pacchi smistati fino a quel momento. Lo stream termina, in maniera esplicita, dopo 10 messaggi.
- **rpc listPackages(DestinationRequest) returns (stream Package)**: restituisce uno stream contenente le informazioni di tutti i pacchi attualmente presenti nelle due code la cui *destination* coincide con quella indicata nella richiesta.

Gate: simula i punti di accettazione dei pacchi. All'avvio sono generati 3 thread, ognuno dei quali rappresenta un gate ed effettua 5 depositi invocando la `deposit` in maniera **sincrona (stub bloccante)**.

L'*id* del pacco è incrementale ed univoco nell'intero sistema, mentre *priority* e *destination* sono scelte in maniera casuale; per verificare la gestione degli errori, la *priority* deve poter assumere occasionalmente un valore non previsto. Tra un deposito ed il successivo il thread attende un tempo di t secondi (con t scelto a caso tra 1 e 3).

Courier: simula i corrieri che ritirano i pacchi. All'avvio sono generati 3 thread, ognuno dei quali effettua 4 richieste di ritiro invocando la *collect* in maniera **asincrona**, cioè senza sospendersi al momento dell'invocazione; il thread recupera il risultato solo in un secondo momento e lo stampa a video. La *priority* della richiesta è scelta a caso; tra una richiesta e la successiva il thread attende un tempo di t secondi (con t scelto a caso tra 2 e 4).

Monitor: client gRPC che osserva il comportamento del sistema. All'avvio invoca la *watchStatus* e stampa a video ogni aggiornamento di stato ricevuto, fino alla chiusura dello stream da parte del server. Successivamente invoca la *listPackages* per una destinazione scelta a caso e stampa i pacchi ricevuti.

Gate, Courier e Monitor devono essere applicazioni eseguibili in maniera indipendente. Si presti attenzione alla corretta sincronizzazione dell'accesso alle code ed ai contatori condivisi, trattandosi di un server concorrente, ed alla corretta terminazione degli stream e alla chiusura dei canali da parte dei client.

File *sorting.proto*

```
syntax = "proto3";

message Empty {}

message Package {
    int32 id = 1;
    string priority = 2;           // "standard" | "express"
    string destination = 3;
}

message Ack {
    string result = 1;
}

message CollectRequest {
    string priority = 1;
    string courier_id = 2;
}

message Status {
    int32 standard_size = 1;
    int32 express_size = 2;
    int32 total_sorted = 3;
}

message DestinationRequest {
    string destination = 1;
}

service SortingService {
    rpc deposit(Package) returns (Ack);
    rpc collect(CollectRequest) returns (Package);
    rpc watchStatus(Empty) returns (stream Status);
    rpc listPackages(DestinationRequest) returns (stream Package);
}
```

Linguaggio. Il sistema può essere realizzato indifferentemente in **Python** o in **Java**. Nella versione Java si utilizzino un *ExecutorService* per il pool di thread del server, lo *blocking stub* per la *deposit* e lo *async stub* (o il *future stub*) per la chiamata asincrona della *collect*, avendo cura di chiudere il *ManagedChannel* in un blocco *finally*. Nella versione Python si utilizzino un *ThreadPoolExecutor*, i *generator* per i metodi in streaming e il metodo *future()* dello stub per la chiamata asincrona.

Estensione facoltativa. Si sostituisca la memorizzazione su file con una persistenza su **MongoDB**: i pacchi prelevati sono salvati come document nella collection *sorted*, ed il metodo *listPackages* recupera i dati direttamente dal database anziché dalle strutture in memoria.